

UDF – Centro Universitario

Ciência da Computação

Ian Cristyan Sousa da Silva

Revisão de Estrutura de Árvores

Brasilia

2026

Ian Cristyan Sousa da Silva

Revisão de Estrutura de Árvores

Atividade apresentada à disciplina de Estruturas de Dados como requisito para avaliação do conteúdo sobre estruturas de árvores.

Brasilia

2026

1 ETAPA 1 — CONCEITOS E ESTRUTURAS DE ÁRVORES

1.1 Conceitos básicos de árvore

Uma árvore é uma estrutura de dados hierárquica formada por nós. Possui uma raiz, que pode ter filhos, e os nós sem filhos são chamados de folhas. É usada para organizar dados de forma hierárquica e facilitar buscas.

1.2 Árvore geral

Cada nó pode ter vários filhos, sem uma quantidade fixa. É útil para representar estruturas hierárquicas, como pastas e sistemas de arquivos.

1.3 Árvore binária

Cada nó pode ter no máximo dois filhos: esquerdo e direito. Serve de base para estruturas como ABB, AVL e rubro-negra.

1.4 Árvore binária de busca (ABB)

Organiza os valores menores à esquerda e os maiores à direita. A busca e a inserção são feitas por comparações. É útil para armazenar dados ordenados e realizar buscas.

1.5 Árvore AVL

É uma ABB que mantém a árvore balanceada. Quando necessário, utiliza rotações para corrigir diferenças de altura entre os lados. É útil quando se busca manter operações rápidas.

1.6 Árvore rubro-negra

É uma ABB que utiliza cores e regras específicas para controlar o balanceamento. Após alterações, pode realizar recolorações e rotações. É usada em estruturas que precisam de buscas, inserções e remoções eficientes.

1.7 Árvore B

Pode armazenar várias chaves em cada nó. Quando um nó fica cheio, ele pode ser dividido. É muito utilizada em bancos de dados e sistemas de arquivos.

1.8 Árvore B+

É semelhante à árvore B, mas concentra os registros ou referências nas folhas, que geralmente são ligadas entre si. Isso facilita buscas e consultas por intervalos, sendo comum em bancos de dados.

1.9 Heap

É uma árvore binária completa organizada por prioridade. No max-heap, o maior valor fica na raiz; no min-heap, o menor. É utilizada em filas de prioridade e no heapsort.

1.10 Trie

Organiza palavras caractere por caractere. Palavras com o mesmo início compartilham os mesmos nós. É usada em autocompletar, dicionários e buscas por prefixo.

2 ETAPA 2 — QUADRO COMPARATIVO

Estrutura	Organização dos dados	Regra ou propriedade	Operação ou ajuste	Aplicação	Referência
Árvore geral	Nós podem possuir qualquer quantidade de filhos.	Não há limite fixo de filhos por nó.	Inserção e remoção alteram os relacionamentos entre pai e filhos.	Sistemas de arquivos e estruturas hierárquicas.	CORMEN et al., Introduction to Algorithms, cap. 10
Árvore binária	Cada nó possui no máximo dois filhos.	Cada nó pode ter filho esquerdo e/ou direito.	Inserção, remoção e alteração de ligações.	Base para outras árvores, como ABB e AVL.	CORMEN et al., Introduction to Algorithms, cap. 12
ABB	Menores à esquerda e maiores à direita.	Esquerda < nó < direita.	Busca, inserção e remoção seguindo comparações.	Busca e armazenamento ordenado.	CORMEN et al., Introduction to Algorithms, cap. 12
AVL	Árvore de busca com altura controlada.	Fator de balanceamento de cada nó entre -1 e 1.	Rotações simples ou duplas após alterações.	Aplicações que exigem buscas rápidas.	CORMEN et al., Introduction to Algorithms, cap. 13
Rubro-negra	Árvore de busca com nós coloridos.	Regras de cores limitam a altura dos caminhos.	Recoloração e rotações.	Mapas, conjuntos e estruturas de busca balanceadas.	CORMEN et al., Introduction to Algorithms, cap. 13
B	Várias chaves podem ficar em um mesmo nó.	Nós respeitam limites de chaves e todas as folhas ficam no mesmo nível.	Divisão, redistribuição e fusão de nós.	Bancos de dados e sistemas de arquivos.	CORMEN et al., Introduction to Algorithms, cap. 18
B+	Chaves de orientação ficam nos nós internos e registros/referências nas folhas.	Todas as folhas ficam no mesmo nível e são normalmente encadeadas.	Divisão e redistribuição de nós.	Índices de bancos de dados e consultas por intervalo.	COMER, The Ubiquitous B-Tree, 1979
Heap	Árvore binária completa organizada por prioridade.	Pai possui prioridade maior ou menor que seus filhos.	Subida e descida de elementos para restaurar o heap.	Filas de prioridade e heapsort.	CORMEN et al., Introduction to Algorithms, cap. 6
Trie	Caracteres são organizados em caminhos compartilhados.	Palavras com o mesmo prefixo compartilham parte do caminho.	Criação/remoção de nós durante inserções e exclusões.	Autocomplete, dicionários e busca por prefixos.	CORMEN et al., Introduction to Algorithms, cap. 34

3 ETAPA 3 — IDENTIFICAÇÃO E JUSTIFICATIVA

3.1 Situação 1 — AVL

A estrutura é uma AVL, pois controla a altura da árvore e utiliza rotações para manter o balanceamento. Limite da analogia: uma estante real não possui balanceamento automático nem rotações controladas por algoritmo.

3.2 Situação 2 — Árvore B

A estrutura é uma árvore B, pois permite várias chaves por nó e pode dividir um nó quando ele fica cheio. Limite da analogia: uma página de catálogo não possui exatamente todas as regras de uma árvore B.

3.3 Situação 3 — Heap

A estrutura é um max-heap, pois mantém o elemento de maior prioridade na raiz para que seja retirado primeiro. Limite da analogia: uma fila comum é linear, enquanto o heap possui organização hierárquica.

3.4 Situação 4 — Trie

A estrutura é uma Trie, pois organiza palavras por caracteres e compartilha os prefixos iguais. Limite da analogia: as letras representam caminhos na estrutura, e não uma sequência física de caracteres.

3.5 Situação 5 — Rubro-negra

A estrutura é uma árvore rubro-negra, pois usa cores, recolorações e rotações para controlar a altura dos caminhos. Limite da analogia: as cores são apenas informações usadas pelo algoritmo e não representam características dos dados.

3.6 Situação 6 — B+

A estrutura é uma árvore B+, pois direciona a busca pelos nós internos até as folhas, que ficam ligadas para facilitar consultas por intervalo. Limite da analogia: um índice de banco de dados possui outros mecanismos além da própria árvore.

3.7 Situação 7 — ABB

A estrutura é uma ABB, pois valores menores ficam à esquerda e maiores à direita, permitindo direcionar a busca por comparações. Limite da analogia: uma ABB comum pode ficar desbalanceada dependendo da ordem de inserção.

REFERÊNCIAS

ADELSON-VELSKII, Georgii M.; LANDIS, Evgenii M. An algorithm for the organization of information. 1962.

COMER, Douglas. The ubiquitous B-tree. ACM Computing Surveys, v. 11, n. 2, p. 121–137, 1979. DOI: 10.1145/356770.356776.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

KNUTH, Donald E. The Art of Computer Programming. Volume 3: Sorting and Searching. 2. ed. Reading: Addison-Wesley, 1998.